

MVC

Le besoin de *design pattern*

Le code d'une application a besoin d'une organisation spécifique. À ce stade de la formation, les projets sur lesquels vous allez travailler vont commencer à prendre de l'ampleur.

Nombreuses pages, fonctionnalités, données : un site web peut très vite devenir une grosse machine. Sans organisation, c'est le chaos. Sans organisation, chaque développeur participant à un projet peut nommer les fichiers comme il l'entend, les placer où il le souhaite, coder comme il en a envie. Et à mesure qu'un tel projet grandit, chaque modification, chaque évolution devient plus fastidieuse et dangereuse que la précédente, notre application devient un château de cartes que plus personne ne comprend.

Nous programmons donc une application en suivant un ensemble de règles architecturales. C'est ce que l'on appelle en programmation un *design pattern*. Organiser une base de code amène un ensemble de problématiques structurelles, aussi avons-nous construit avec le temps des conventions et des structures que nous nous imposons dans notre travail afin de répondre en amont à ces problématiques. On ne code alors plus n'importe comment, on obéit à une certaine structure-type.

Les bases d'un site web

Rappelons le fondement d'un site web, dans son fonctionnement traditionnel et courant :

Le client – au sens « utilisateur » du terme – effectue une requête auprès du serveur. « Je veux afficher la page de contact. » Le serveur réceptionne cette requête et s'organise intérieurement pour y répondre. Il doit analyser l'URL demandée pour savoir quelle page afficher, éventuellement communiquer avec MySQL pour récupérer des données si nécessaire, et en finalité générer un document HTML qu'il envoie au client.

Construire ce document HTML et l'envoyer en réponse au navigateur de l'utilisateur, c'est le but du jeu du web. Cette construction peut être plus ou moins complexe en fonction de la page ou de l'action demandée, complexité qui demande une certaine organisation. Comment agencer nos fichiers, comment les faire communiquer entre eux, comment nommer nos modules. Bref, un *design pattern*.

MVC

Il existe de nombreux *design patterns* en programmation. Dans le monde du web, et plus précisément dans le monde du PHP, le plus courant est le **MVC**, pour « *Model – View – Controller* ». C'est celui que vous rencontrerez le plus fréquemment dans une carrière classique de développeur PHP, et c'est notamment celui auquel obéissent les deux principaux *frameworks* du marché du travail : Symfony et Laravel.

Pourquoi le MVC plutôt qu'un autre ? Parce que c'est le design qui découle le plus naturellement des problématiques amenées par l'écosystème du développement web PHP et son histoire.

Un site web, c'est l'agencement de trois grandes composantes :

- L'interface visuelle ;
- Les traitements logiques ;
- La gestion des données.

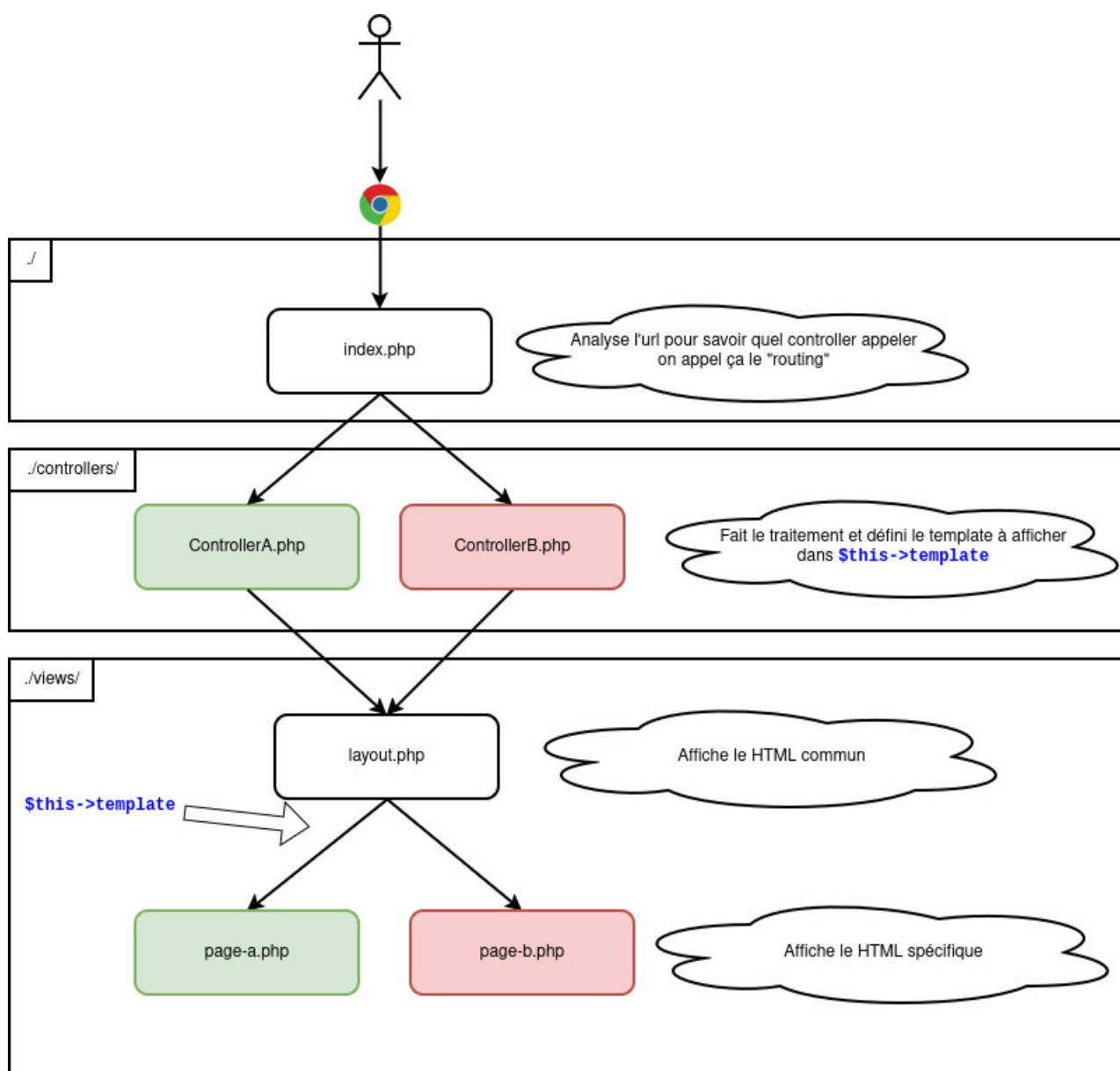
S'imposer la règle de structurellement séparer ces trois composantes dans l'organisation d'une application est le paradigme de départ du design MVC.

Views et Controllers

Vues et Contrôleurs

Vous rencontrerez de nombreuses interprétations et applications du **MVC**, mais celles-ci se ressembleront toujours dans les grandes lignes.

Mettons pour l'instant le **Model** de côté. Concentrons-nous sur l'interaction entre les vues et les contrôleurs. Nous vous proposons ici un schéma très classique de l'organisation d'un projet web :



Ici nos **Controllers** sont des objets. Considérant que chaque flèche représente un **require** PHP, détaillons le fonctionnement de ce design.

Point d'entrée

Le fichier **index.php** est le point d'entrée de **TOUTES** nos requêtes.

L'**index.php** s'occupe d'inclure les fichiers nécessaires, de démarrer la session, et de faire ce que l'on appelle le **routing**, c'est à dire choisir quel contrôleur appeler.

Le routing

Le **routing** consiste à choisir quel contrôleur devra traiter la requête. La forme la plus simple de routing est l'utilisation d'une variable **\$_GET** (souvent **page**) qui indiquera par un mot clé le contrôleur à utiliser.

Par exemple, l'URL **index.php?page=director** appellera le contrôleur **DirectorController**, tandis que **index.php?page=home** appellera le contrôleur **HomepageController**. Selon la variable **\$_GET['page']** notre **index.php** viendra donc instancier un contrôleur du type voulu.

Généralement en plus d'instancier le contrôleur, **index.php** appellera aussi une méthode de ce contrôleur, dans notre cas nous choisirons de nommer cette méthode **display**.

Contrôleur et traitement logique

Maintenant que nous avons choisi le contrôleur à appeler, c'est celui-ci qui va devoir assurer le traitement logique de la page.

Il va par exemple définir des variables, qu'il pourra passer ensuite au HTML au moment de l'affichage.

Le but du contrôleur sera toujours le même, faire les traitements logiques, préparer les données à afficher, définir le template à utiliser (dans notre cas il stockera cette donnée dans **\$this->template**) et inclure **layout.php**.

Rappel sur layout :

Le *layout* représente le squelette de notre HTML. Les parties communes à toutes les pages et inclus au milieu le *template* précédemment défini : `require ($this->template);`.

Le voyage est terminé, nous avons généré le HTML.

En résumé :

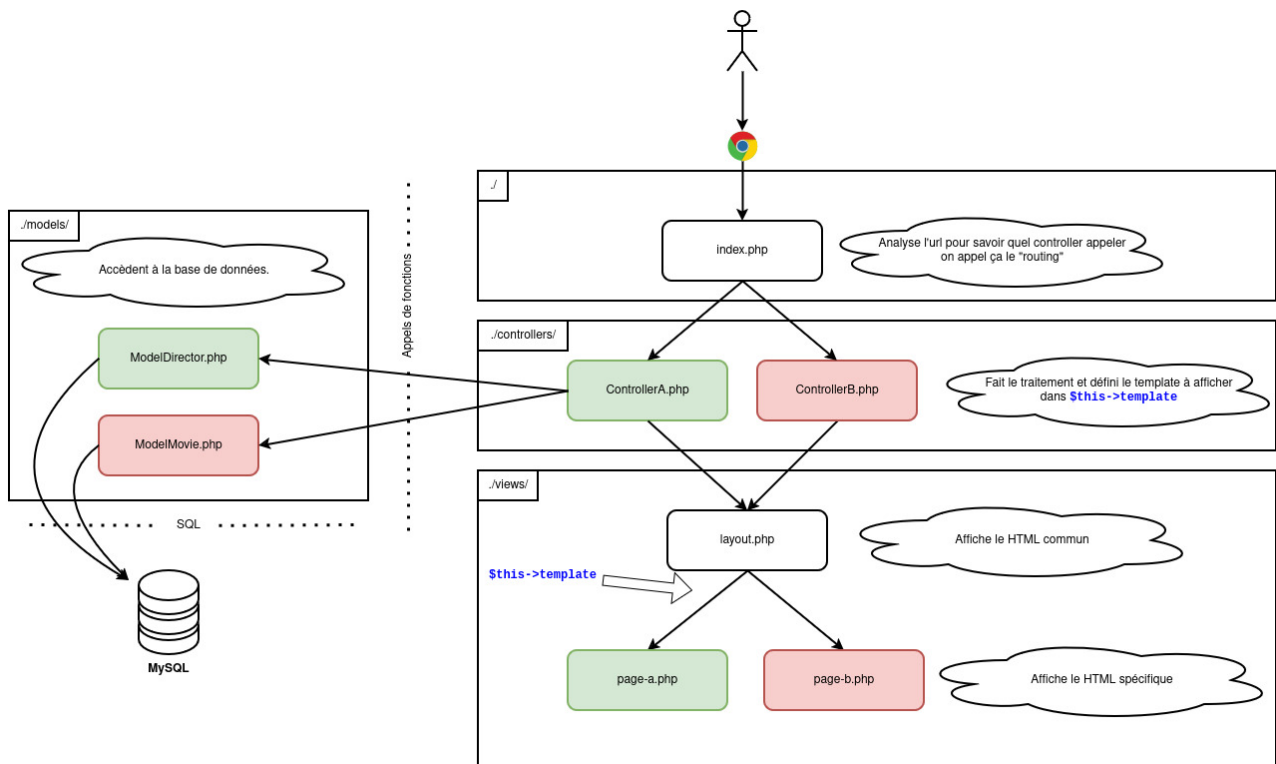
- L'`index.php` effectue le *routing* et appelle notre *controller*;
- Le *controller* définit le *template* à utiliser et appelle le *layout* ;
- Le *layout* appelle le *template* précédemment défini.

Models

Models

Nos contrôleurs gèrent la logique, nos vues l'affichage, nos modèles vont être en charge de l'accès aux données.

Reprenons notre schéma **MVC** et ajoutons les modèles.



Nous voyons que nos modèles servent de lien entre nos contrôleurs, et la base de données. Plutôt que de faire les requêtes SQL directement, nos contrôleurs vont appeler **les fonctions exposées par les modèles**, qui eux feront les requêtes SQL.

Un modèle par table

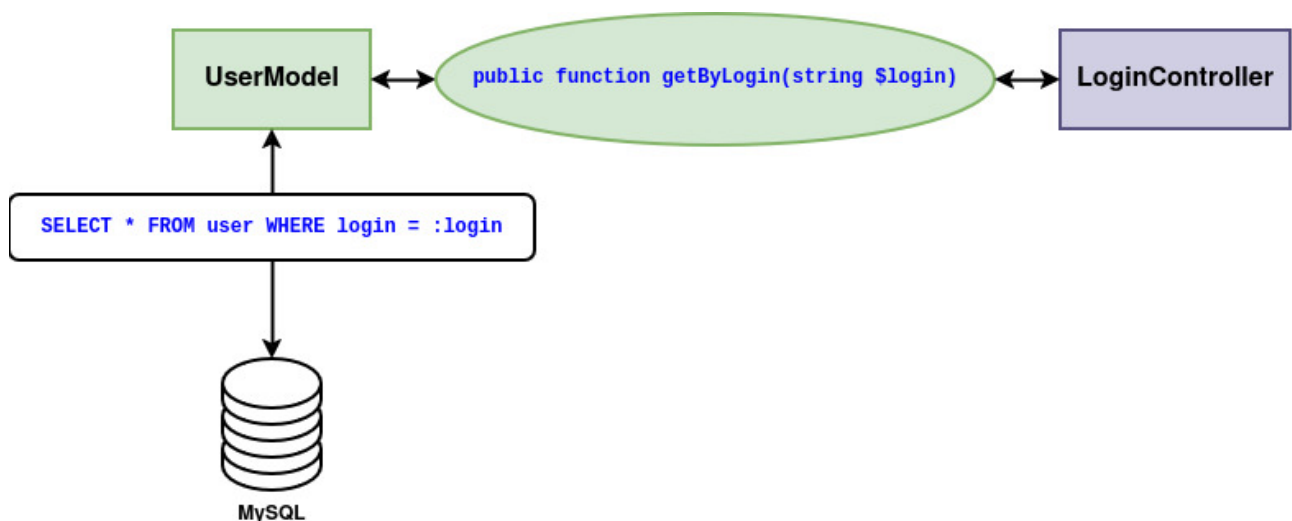
Pour chaque table de notre base de données (en dehors des tables de liaisons) nous devrions créer un modèle correspondant. C'est ce modèle qui sera chargé de faire les requêtes sur cette table.

Relation entre modèle et contrôleur

Si nous reprenons le schéma précédent nous voyons que les modèles sont un peu excentrés du flux principal et ne communiquent qu'avec les contrôleurs.

Concrètement un contrôleur peut faire appel à n'importe quel modèle pour faire des requêtes sur la base de données. Les modèles vont donc exposer un certain nombre de méthodes publiques (généralement une méthode par requête vers la base), et ce sont ces méthodes qu'appelleront les contrôleurs.

Vous pouvez donc voir les modèles comme une sorte de passe plat, de traducteur entre d'un côté PHP, avec des fonctions et des paramètres, et de l'autre la base de données avec **PDO** et des requêtes SQL.



Aller plus loin avec les entités

Dans le cadre de notre cours nous nous contenterons de récupérer les objets comme des tableaux associatifs, mais avec des modèles plus poussés nous pourrions transformer ces tableaux en véritables objets PHP représentant les lignes de notre base de données.

On parle alors d'ORM, pour Object-Relationnal Mapping, et vous trouverez souvent l'objet en question désigné sous le nom **d'entité**.

De tels systèmes sont disponibles dans tous les frameworks majeurs, Symfony, Laravel, etc.

Nous en resteront à de simples tableaux associatifs pour le moment, mais il est bon de savoir que des systèmes plus complexes existent et de connaître au moins le vocabulaire et les grandes notions associées !

Namespace et Autoloader

Les namespaces pour séparer nos classes

Par défaut, il n'est pas possible de créer deux classes avec le même nom dans notre PHP.

```
// Nous sommes bien d'accord qu'il est IMPOSSIBLE de  
// faire ceci :
```

```
class Utilisateur {  
    public function sayHello() {  
        return "Bonjour!";  
    }  
}
```

```
class Utilisateur {  
    public function sayHi() {  
        return "Salut!";  
    }  
}
```

```
// 2 classes ne peuvent avoir le même nom dans le  
// même espace de noms.
```

Cette limite est bien logique, si les deux classes ont le même nom, comment PHP pourrait-il savoir laquelle nous appelons quand nous faisons `new Utilisateur()` ?

Seulement voilà, si le projet devient suffisamment grand, il y a bien un moment où nous finirons par vouloir utiliser un nom qui a déjà été employé ailleurs... C'est là que les namespaces (espaces de noms en français) viennent nous sauver !

Les namespaces nous permettent de regrouper des classes, fonctions et constantes, d'organiser le code et surtout d'éviter les conflits de nom.

Grâce au namespaces, nous allons pouvoir dire à PHP "ici tu es dans la boîte Boite1, et toutes les classes, fonctions, etc. suivantes seront dans cette boîte".

Avec namespace :

```
namespace LibraryA;
```

```
class Utilisateur {  
    public function sayHello() {  
        return "Bonjour!";  
    }  
}
```

```
namespace LibraryB;
```

```
class Utilisateur {  
    public function sayHi() {  
        return "Salut!";  
    }  
}
```

```
}
```

Grâce à l'utilisation des namespaces nous pouvons donc désormais créer deux classes avec le même nom, car si leur nom local est le même, leur nom complet, (namespace + nom local) est différent.

Grâce aux namespaces, nous obtenons deux classes '**Utilisateur**' dans des espaces de noms différents.

Comment les utiliser ?

```
$userA = new LibraryA\Utilisateur();  
echo $userA->sayHello();
```

```
$userB = new LibraryB\Utilisateur();  
echo $userB->sayHi();
```

```
// On peut utiliser l'aliasing pour abréger les namespaces
```

```
use LibraryA\Utilisateur as UserA;
```

```
$user = new UserA();
```

Les namespaces s'utilisent pour les **classes**, les **fonctions**, les **constantes**.

Ils doivent être déclarés tout en haut du fichier, avant toute autre chose (à part les déclarations '**declare**').

L'autoloader

Les namespaces c'est bien, mais ça oblige à venir importer pleins de fichiers ou à écrire des noms à ralonger. Pour nous simplifier la vie PHP propose un mécanisme qui permet d'inclure automatiquement les fichiers quand on crée une nouvelle classe, l'autoloader.

L'autoloader est un mécanisme qui permet de charger **automatiquement** des classes lorsqu'on les utilise, sans avoir à les inclure manuellement par un '**include**' ou '**require**'. Essentiel lorsqu'on travaille avec beaucoup de classes dans beaucoup de fichiers !

1. Exemple simple

Voici l'arborescence de notre projet.

```
/projet  
  /classes  
    Utilisateur.php  
    Produit.php  
  index.php
```

Il existe une fonction php `spl_autoload_register` qui se lance automatiquement dès qu'on instancie une classe. Il suffit alors d'indiquer à cette fonction où trouver la classe à instancier.

Dans `index.php` pour inclure toutes nos classes, on aurait :

```
// $classname contient le nom de la classe que l'on essaye d'instancier
spl_autoload_register(function ($classname) {
    // on inclut le fichier du même nom que la classe ayant l'extension .php et se trouvant dans le dossier classes
    include 'classes/' . $classname . '.php';
});
```

```
// l'instanciation provoque ainsi un include de "classes/Utilisateur.php"
$user = new Utilisateur();
$product = new Produit();
```

Au moment où la classe `Utilisateur` est instanciée, si PHP ne connaît pas encore la classe il essaiera d'inclure le fichier `classes/Utilisateur.php` avant d'instancier cette classe.

2. Exemple dans un contexte MVC

Pour résumer le rôle de l'autoloader, on pourrait dire que **son rôle est de transformer des namespaces en chemins de fichiers.**

Pour qu'il puisse remplir son rôle, nous allons donc devoir créer une fonction en charge de transformer les namespaces en chemins de fichier, et l'enregistrer comme autoloader.

Enregistrement de l'autoloader

Il va falloir indiquer à notre autoloader comment aller chercher les fichiers qui contiennent **les classes** que l'on souhaite instancier.

Admettons que l'on souhaite utiliser la classe `User` qui se trouve parmi nos modèles, dans le fichier `src/Models/User.php`, et que nous souhaitons pouvoir charger automatiquement ce fichier si nous utilisons l'instruction ci-dessous.

```
use App\Models\User;
```

Nous allons donc créer un fichier `autoload.php` avec le code ci-dessous, que nous incluerons dans notre fichier `index.php` avant toute autre action :

```
spl_autoload_register(function ($classname) {  
    // $classname représente la classe que le programme souhaite  
    // instancier avec l'ensemble de ses namespace, ici "App\Models\User"  
    // On remplace les backslashes du namespace par des slashes de  
    // chemin de dossier, ce qui nous donne : "App/Models/User"  
    $path = str_replace('\\', '/', $classname);  
  
    // On remplace "App" par "src" dans le chemin, ce qui nous  
    // donne : "src/Models/User"  
    $path = str_replace('App', 'src', $path);  
  
    // On ajoute l'extension .php, ce qui nous donne : "src/Models/  
    // User.php"  
    $filename = $path . '.php';  
  
    // On inclut le fichier si il existe  
    if (file_exists($filename)) {  
        include $filename;  
    }  
});
```

Utilisation d'une classe

Ainsi, partout dans notre code, dans un controller par exemple, si l'on instancie la classe `User` après avoir mis en haut du fichier concerné un `use App\Models\User`, PHP va automatiquement appeler la fonction qu'on aura préalablement enregistrée grâce à `spl_autoload_register()` et chargera le fichier correspondant.

Dès que nous avons besoin d'utiliser un contrôleur, un modèle ou une vue dans notre code, nous n'avons pas à nous inquiéter de l'inclure. L'autoloader s'en occupe pour nous, grâce à la convention de nommage des namespaces et des fichiers.

PSR-4 ?

Il y a une convention stricte sur la façon dont les namespaces doivent correspondre aux chemins de fichiers. Lorsqu'on utilise une classe, l'autoloader regarde son namespace, le convertit en chemin de fichier en suivant les règles de **PSR-4**, et inclut le fichier approprié.

Il existe un groupe de personnes dévouées à l'interopérabilité entre différents projets PHP, le groupe PHP-FIG (Framework Interop Group). Ils ont établi la norme PSR-4, recommandée notamment dans le travail d'équipe.

Vous pouvez lire toutes leurs recommandations ici : [PHP-FIG](#)

Mise en place d'une architecture VC

Rappels MVC

Modèle : Gère les données et la logique métier.

Vue : Présente les données à l'utilisateur.

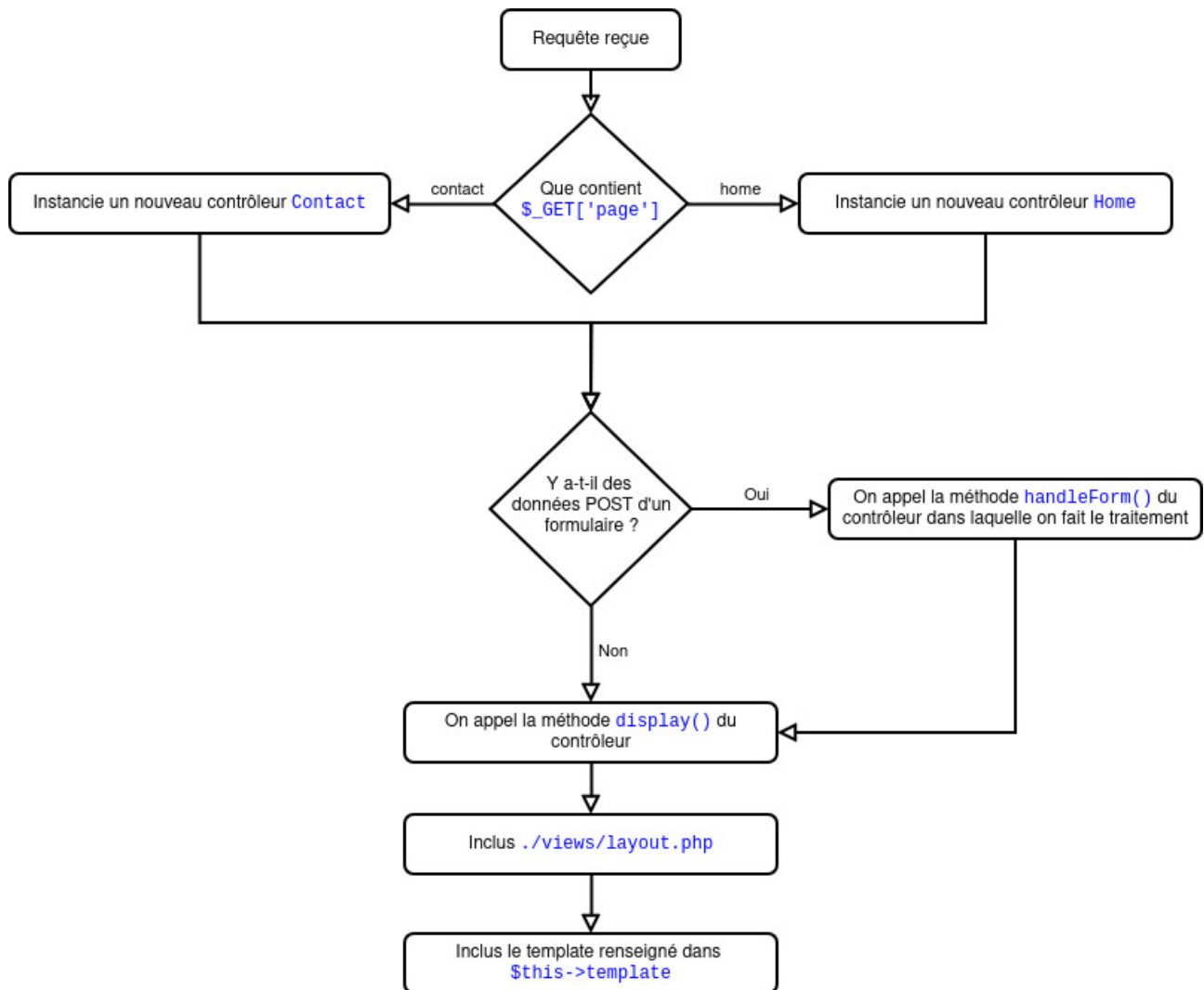
Contrôleur : Fait le lien entre le modèle et la vue, gère les interactions.

Fonctionnement général de notre MVC

Avant de voir un peu plus en détails le fonctionnement interne de notre MVC, nous allons commencer par une petite série de schéma illustrant le fonctionnement global de notre application et de notre "micro-framework" MVC, le parcours logique d'une requête et les liens entre contrôleurs et modèles.

Parcours logique d'une requête

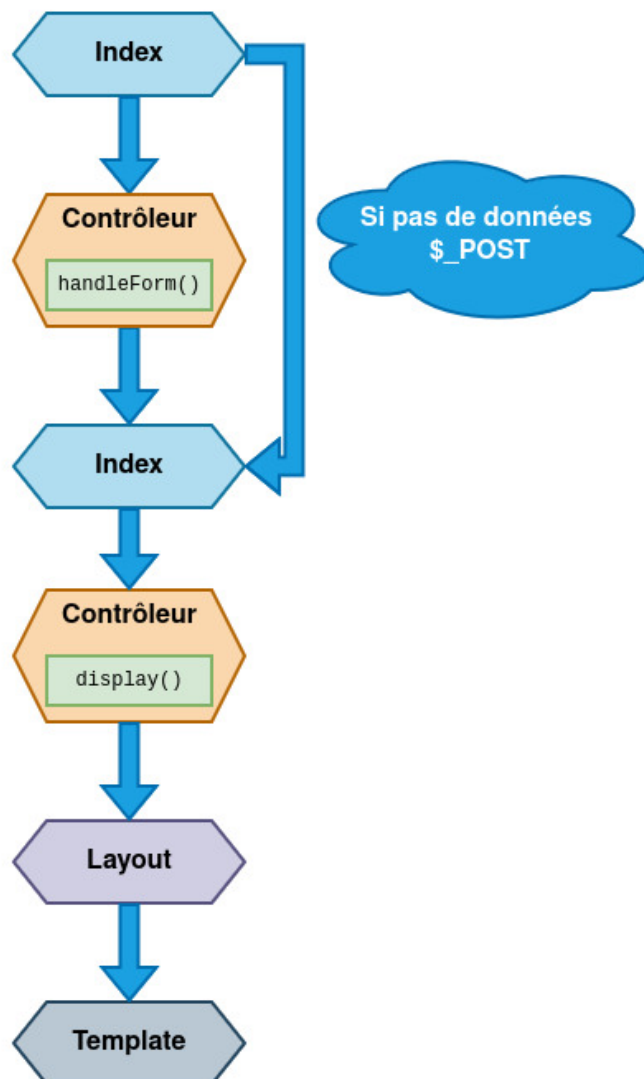
Le schéma ci-dessous vous permet de mieux visualiser le chemin logique du traitement d'une requête reçue par notre MVC.



Ordre de traitement d'une requête (callstack)

Et ce schéma devrait vous permettre de mieux comprendre quand quel fichier est traité chaque partie !

Ordre de traitement d'une requête dans le modèle MVC



Structure des Dossiers

Dans un premier temps nous allons créer un dossier pour notre projet de façon à séparer nos **models**, **views** et **controllers**, que nous mettrons dans un dossier **src** ainsi qu'un dossier public pour stocker les **ressources** (CSS, images, JS, fonts...).

Nous aurons également des fichiers **index.php**, **autoload.php** et **connection.php** à la racine.

```
MVC_Project/
|-- src/
|   |-- controllers/
|   |   |-- AbstractController.php
|   |   |-- Home.php
|   |   |-- Movie.php
|   |-- models/
|   |   |-- AbstractModel.php
|   |   |-- Movie.php
|   |-- views/
|   |   |-- home.php
|   |   |-- login.php
|   |   |-- layout.php
|-- public/
|   |-- css/
|   |   |-- style.css
|   |-- js/
|   |   |-- script.js
|-- index.php
|-- autoload.php
|-- connection.php
```

Création du routeur **index.php**

Dans une architecture **MVC**, le fichier **index.php** est le point d'entrée de toute page.

Notre fichier **index.php** sera donc toujours appelé en premier et devra, en fonction des paramètres qui lui seront transmis (pour nous **\$_GET['page']**), appeler le controller adapté.

Exemple d'index avec un paramètre d'url **page** :

```
<?php
session_start();

require('./autoload.php');
require('./connection.php');

// On cherche la page à utiliser, et on utilise home par défaut
if (isset($_GET['page'])) {
    $page = $_GET['page'];
} else {
    $page = 'home';
}

// Ici on route selon la page vers le bon controller
switch ($page) {
    case 'home':
        $controller = new \App\Controllers\Home();
        $controller->display();
        break;
    case 'movie':
        $controller = new \App\Controllers\Movie();
        $controller->display();
        break;

    default:
        echo "Error 404";
}
```

Développement des Contrôleurs

Tous nos contrôleurs hériteront d'un contrôleur abstrait, `AbstractController`, qui sera responsable de fournir le comportement par défaut d'un contrôleur, à savoir :

- Définir une propriété `template`.
- Définir une fonction `display()` qui affichera le **layout**.
- Définir une fonction `handleForm()` qui sera appelée si des données POST sont reçues.

Dans controllers/AbstractController.php :

```
<?php

namespace App\Controllers;

// Notre controller pour la page de contact
abstract class AbstractController
{
    // Des variables pour stocker le template à afficher, les
    // messages d'erreur ou de succès
    public ?string $template = null;
    public ?string $error = null;
    public ?string $message = null;

    // Joue la fonction handleForm si $_POST n'est pas vide
    public function __construct()
    {
        if (!empty($_POST)) {
            $this->handleForm();
        }
    }

    // Affiche notre page de contact
    public function display()
    {
        require('./src/Views/layout.php');
    }

    // La fonction qui gèrera la réception d'un formulaire $_POST
    public function handleForm()
    {
        // By default do nothing
        return;
    }
}
```

Quand nous ferons un nouveau contrôleur nous devrons donc étendre **AbstractController**, puis définir le template à utiliser pour ce contrôleur, ainsi que créer nos propres fonctions display() pour préparer les données nécessaires à l'affichage, ainsi qu'ajouter une fonction handleSubmit si nous devons traiter un formulaire :

```
<?php
```

```
namespace App\Controllers;
```

```
// Notre controller pour la page de contact
```

```
class Contact extends AbstractController
```

```
{
```

```
    public ?string $template = './src/Views/contact.php';
```

```
    // Affiche notre page de contact
```

```
    public function display()
```

```
    {
```

```
        require('./src/Views/layout.php');
```

```
    }
```

```
    // Gère la soumission du formulaire de contact
```

```
    public function handleForm()
```

```
    {
```

```
        // Traitement du formulaire
```

```
        if (empty($_POST['message']) || empty($_POST['email'])) {  
            $this->error = 'Veuillez renseigner un email et un
```

```
message.';
```

```
            return;
```

```
        }
```

```
        // ... Imaginez un traitement
```

```
        // Pour pouvoir stocker des données qui seront affichées  
dans notre template
```

```
        // il faut que nous les ajoutions à notre controller
```

```
        $this->message = 'Votre message a bien été envoyé !';
```

```
    }
```

```
}
```

Développement des Vues

Dans `views/contact.php`, nous trouverons le code propre à la page de contact :

```

<form action="?page=contact" method="post">
    <?php if ($this->error) { ?>
        <p class="alert error"><?= htmlspecialchars($this->error); ?></p>
    <?php } ?>

    <?php if ($this->message) { ?>
        <p class="alert success"><?= htmlspecialchars($this->message); ?
    ></p>
    <?php } ?>

    <label for="email">Email :</label>
    <input type="email" id="email" name="email" required>

    <label for="message">Message :</label>
    <textarea id="message" name="message" rows="4" required></textarea>

    <button type="submit">Envoyer</button>
</form>

```

Dans le fichier `layout.php`, nous trouverons par contre le code qui est commun à l'ensemble des pages (head, header, footer...)

Développement des Modèles

Comme pour les contrôleurs, les modèles hériteront tous d'un ancêtre commun, `AbstractModel`, qui sera en charge de récupérer l'accès à la base de données auprès de `PDO`, et de la rendre disponible auprès des autres modèles dans la propriété `$this->db`. Nous ne détaillerons pas ce code car il n'est pas très intéressant, vous pourrez regarder directement dans les sources.

Nos modèles viendront donc étendre `AbstractModel` et définir une fonction pour chaque requête que nous souhaitons faire auprès de la base de données.

Par exemple, si j'ai une table `movie` et que je veux pouvoir récupérer tous les films, ou bien un film par son id, voilà à quoi à quoi pourra ressembler notre modèle `Movie`.

```
<?php
```

```
namespace App\Models;
```

```
class Movie extends AbstractModel
```

```
{
```

```
    // Une fonction qui va retourner la liste de tous les films
```

```
    public function getAll()
```

```
    {
```

```
        $query = $this->db->prepare('SELECT * FROM movie');
```

```
        $query->execute();
```

```
        return $query->fetchAll();
```

```
    }
```

```
    // Une fonction qui va retourner un film selon son id
```

```
    public function getById(int $id)
```

```
    {
```

```
        $query = $this->db->prepare('SELECT * FROM movie WHERE id = :id');
```

```
        $query->execute(['id' => $id]);
```

```
        return $query->fetch();
```

```
    }
```

```
}
```

Quand nous aurons besoin de récupérer la liste de tous les films, par exemple sur la page d'accueil, il nous suffira de modifier la fonction display du contrôleur associé, d'appeler notre modèle **Movie** et d'utiliser la méthode **getAll()** !

```
<?php
```

```
namespace App\Controllers;
```

```
// Notre controller pour la page d'accueil
```

```
class Home extends AbstractController
```

```
{
```

```
    public ?string $template = './src/Views/home.php';
```

```
    // Affiche notre page d'accueil
```

```
    public function display()
```

```
    {
```

```
        // Je récupère la liste de tous les films
```

```
        $model = new \App\Models\Movie();
```

```
        $movies = $model->getAll();
```

```
        require('./src/Views/layout.php');
```

```
    }
```

```
}
```

Sources

Plutôt que de détailler chaque petite partie de notre système MVC, voici un projet "de base" que vous pourrez adapter pour votre utilisation.

Vous pouvez partir de ce projet, mettre à jour les informations de connection à la base dans `connection.php`, et partir du code des controllers, `models`, etc. pour vous mettre le pied à l'étrier !

Projet - Distorsion

Distorsion

Notre objectif va être de mettre au point un système de messagerie ressemblant fortement à des logiciels célèbres tels que Slack ou Discord.

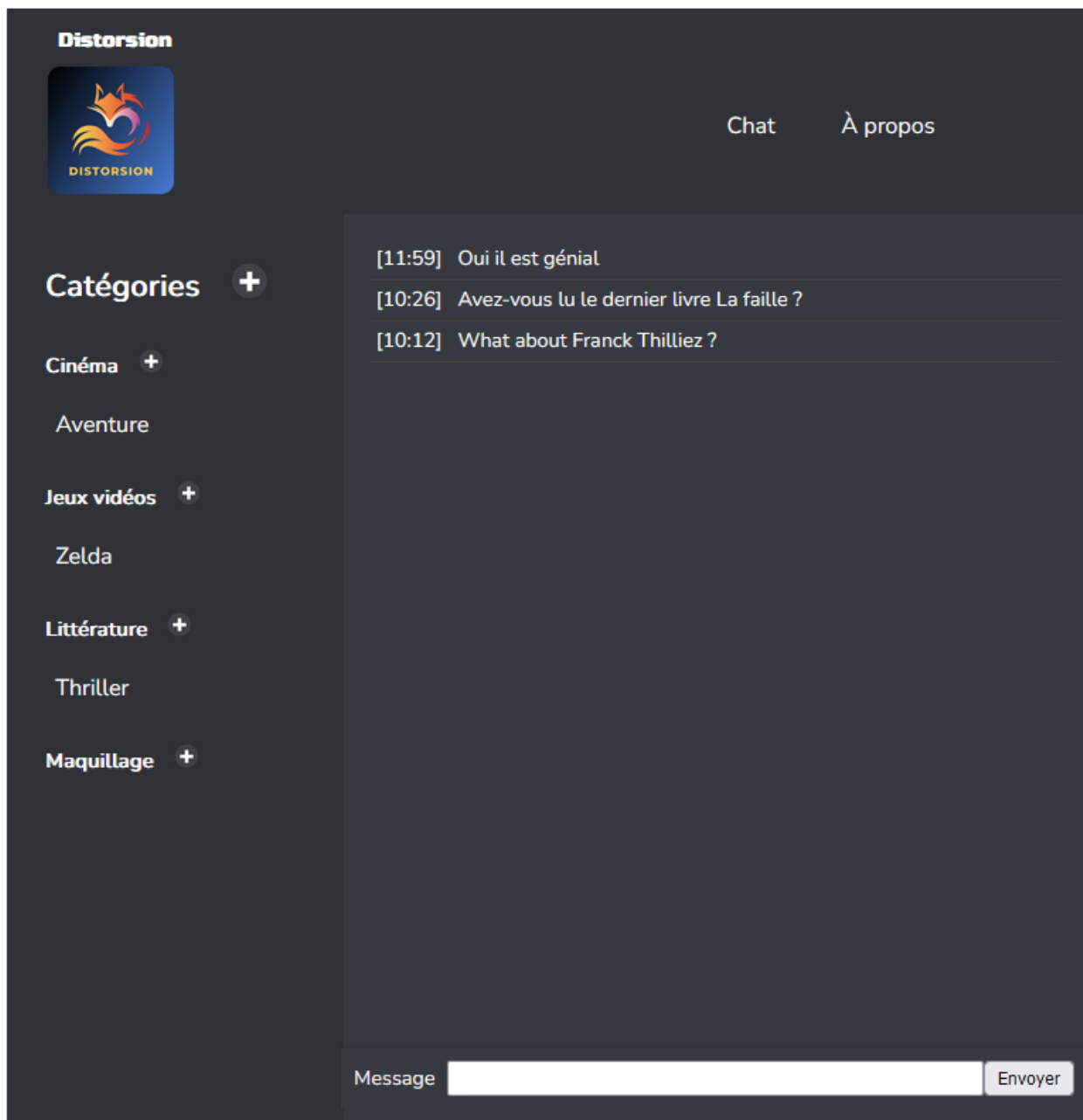
Le principe est simple : les messages sont affichés dans des salons distincts, lesquels sont rangés dans des catégories.

La particularité de notre projet, Distorsion, est qu'il ne sera pas nécessaire de créer un compte pour communiquer. Tous les messages sont anonymes.

Les fonctionnalités sont donc les suivantes :

- Créer des catégories ;
- Créer des salons ;
- Afficher la liste des catégories et de leurs salons respectifs avec la possibilité de naviguer d'un salon à l'autre ;
- Afficher les messages du salon sélectionné, ordonnés par date et heure d'envoi ;
- Envoyer des messages dans un salon ;

Voici à quoi cela peut ressembler :



Contraintes techniques :

- Pour chaque table de la base de données, un model.
- L'architecture doit être en **MVC** avec un controller par page.

Bonus:

- Utiliser Ajax pour mettre à jour les messages en temps (quasi) réel.